



**TECNOLÓGICO
NACIONAL DE MÉXICO**



TOPICOS DE IA

TEMA:

Optimización de Riego con Enjambre de Partículas

Alumnos:

Alfonso Lara Torres

Jose Enrique Felix Esparragoza

Profesor:

Zuriel Dathan Mora Felix

Descripción del problema

El presente proyecto aborda un problema aplicado al sector agrícola relacionado con la optimización del riego en campos de cultivo mediante la ubicación eficiente de sensores. La gestión del agua es un factor crítico en la producción agrícola, especialmente en regiones donde las condiciones ambientales y del terreno presentan variaciones que afectan la retención de humedad y la eficiencia del riego, considerando que el sector agrícola global es el mayor consumidor de recursos de agua dulce, representando aproximadamente el 70% del uso total mundial (Zhang et al., 2024).

El problema consiste en determinar la mejor distribución posible de sensores dentro de un área de cultivo, considerando variables como humedad del suelo, elevación, salinidad, temperatura y tipo de cultivo, con el fin de optimizar la eficiencia del riego y garantizar una adecuada toma de decisiones en la gestión del agua.

En particular, la región de Guasave, Sinaloa, se caracteriza por una llanura costera con altitudes entre 10 y 50 metros sobre el nivel del mar, suelos fértiles pero con zonas de salinidad y erosión superficial, y una producción agrícola diversa que incluye principalmente maíz, tomate y chile. Esta variabilidad en las condiciones del terreno y los cultivos influye directamente en los niveles de humedad, lo que hace necesario emplear un enfoque de optimización inteligente que permita adaptar la ubicación de los sensores a las características específicas de cada zona.

Por ello, se plantea resolver la distribución óptima de sensores mediante el algoritmo bioinspirado de Enjambre de Partículas (PSO), con el propósito de encontrar las posiciones que maximicen la eficiencia del riego.

Justificación del uso de PSO

El algoritmo bioinspirado de Enjambre de Partículas (PSO, por sus siglas en inglés: Particle Swarm Optimization) representa una técnica de optimización metaheurística eficiente para la resolución de problemas de optimización continua y multidimensional, características que se ajustan a la naturaleza del problema de distribución óptima de sensores de humedad en terrenos agrícolas. PSO no requiere suposiciones sobre la continuidad y diferenciabilidad de la función objetivo a optimizar, no necesita calcular el gradiente de la función de error, y no requiere puntos de inicio apropiados ni conocimiento profundo a priori sobre las áreas más prometedoras del espacio de búsqueda (Marini & Walczak, 2015).

PSO se basa en el comportamiento colectivo de sistemas naturales, como el movimiento coordinado de bandadas de aves o cardúmenes de peces, donde cada individuo ajusta su posición en función de su experiencia y la del grupo. Esta dinámica le permite explorar de manera eficiente el espacio de soluciones y converger hacia regiones óptimas sin necesidad de derivadas u operaciones complejas.

En el contexto del presente proyecto, PSO resulta especialmente adecuado porque:

- Optimización en espacios continuos: PSO es particularmente adecuado para problemas de optimización con espacios de búsqueda continuos, lo que corresponde directamente al problema de ubicación de sensores donde las coordenadas son valores continuos (Zhang et al., 2015).
- Simplicidad de implementación y pocos parámetros: PSO presenta la ventaja principal de tener menos parámetros que ajustar en comparación con otros algoritmos evolutivos, facilitando su aplicación práctica en entornos agrícolas reales (Bonyadi & Michalewicz, 2022).
- Balance entre exploración y explotación: El movimiento de cada partícula está influenciado por su mejor posición local conocida y por las mejores posiciones del enjambre, lo que permite explorar ampliamente el espacio de búsqueda mientras explota regiones prometedoras de manera eficiente (Bonyadi & Michalewicz, 2022).
- Aplicaciones exitosas en redes de sensores: PSO ha sido aplicado exitosamente en problemas de redes de sensores inalámbricos tales como despliegue óptimo, localización de nodos y agregación de datos, demostrando ser un algoritmo simple, efectivo y computacionalmente eficiente (Kulkarni & Venayagamoorthy, 2011). Además, ha demostrado efectividad en la optimización de cobertura de sensores para maximizar la cobertura y minimizar la redundancia (Dafan et al., 2025).

Por estas razones, el algoritmo PSO representa una alternativa eficiente, adaptable y de bajo costo computacional para la optimización de la ubicación de sensores de humedad, contribuyendo así a mejorar la eficiencia del riego y el uso sostenible del recurso hídrico en la región de Guasave, Sinaloa.

Diseño del algoritmo

La implementación del proyecto se realizó en Python, debido a su ecosistema robusto de bibliotecas especializadas para computación científica, procesamiento de datos y algoritmos de optimización. Python ofrece herramientas maduras y bien documentadas que facilitan tanto el desarrollo como la experimentación con técnicas de inteligencia artificial, siendo el lenguaje estándar para proyectos de machine learning y optimización metaheurística.

Las bibliotecas utilizadas fueron:

- Pandas y NumPy: Constituyeron la base para la manipulación y procesamiento de datos. Pandas se utilizó para cargar el archivo CSV en DataFrames y facilitar el manejo de datos tabulares, mientras que NumPy permitió realizar operaciones matriciales eficientes, fundamentales para los cálculos de distancias y la evaluación de la función de costo.
- PySwarms: Fue la herramienta central para la implementación del algoritmo PSO. Esta librería proporciona implementaciones optimizadas y probadas de diversas variantes de PSO, incluyendo GlobalBestPSO utilizado en este proyecto. PySwarms abstrae la complejidad del algoritmo, permitiendo enfocarse en la definición del problema (función de costo y espacio de búsqueda) mientras maneja automáticamente la actualización de velocidades, posiciones y el seguimiento del mejor global.
- Scikit-learn: Se empleó específicamente el módulo MinMaxScaler para normalizar todas las características al rango [0, 1]. Esta normalización es crítica para PSO, ya que el algoritmo es sensible a las escalas de los datos y variables con rangos mayores podrían dominar el proceso de optimización.
- SciPy: La función cdist del módulo scipy.spatial.distance permitió calcular eficientemente matrices de distancias euclidianas entre todos los puntos de datos y las ubicaciones de sensores propuestas, optimizando significativamente el rendimiento de la función de costo.
- Matplotlib y Seaborn: Se utilizaron para generar visualizaciones de los resultados. Matplotlib permitió crear gráficos de convergencia del algoritmo, mientras que Seaborn facilitó la generación de mapas de dispersión estilizados para visualizar la ubicación de cultivos y sensores óptimos.

Una vez establecidas las herramientas para la gestión de datos y la optimización, el núcleo del algoritmo PSO se implementó en la clase OptimizadorSensoresPSO de manera modular. El proceso se estructuró en una serie de métodos interconectados, cada uno con una responsabilidad específica dentro de la lógica del programa.

Inicialización del Optimizador:

El constructor de la clase OptimizadorSensoresPSO recibe todos los parámetros necesarios para configurar el problema de optimización y calcula las dimensiones del espacio de búsqueda. El número total de dimensiones se define como el producto del número de sensores por el número de características, ya que cada sensor debe posicionarse en el espacio multidimensional de características.

```
def __init__(self, n_sensores, datos, n_particulas, iteraciones, escalador,
columnas_caracteristicas):
    self.n_sensores = n_sensores
    self.datos = datos
    self.n_particulas = n_particulas
    self.iteraciones = iteraciones
    self.escalador = escalador
    self.columnas_caracteristicas = columnas_caracteristicas

    # Definir dimensiones del problema
    self.n_caracteristicas = datos.shape[1]
    self.n_dimensiones = self.n_sensores * self.n_caracteristicas

    self.mejor_costo = np.inf
    self.mejor_posicion = None
    self.historial_costos = []
```

_funcion_costo_particula:

El método `_funcion_costo_particula` es el corazón del algoritmo de optimización. Define cómo se evalúa la calidad de una configuración de sensores propuesta. La función reforma el vector de posición de la partícula en una matriz donde cada fila representa un sensor con todas sus características. Luego, calcula las distancias euclidianas entre todos los puntos de datos y todos los sensores propuestos, identifica para cada punto de dato cuál es el sensor más cercano, y finalmente suma las distancias al cuadrado como medida del costo total. Un costo menor indica que los sensores propuestos cubren mejor todos los puntos del campo agrícola.

```
def _funcion_costo_particula(self, posicion_particula):
    try:
        centroides = posicion_particula.reshape(self.n_sensores,
self.n_caracteristicas)
        except ValueError as e:
            print(f"Error al reformar: {e}")
            return np.inf

        distancias = cdist(self.datos, centroides, 'euclidean')
        min_distancias = np.min(distancias, axis=1)
        costo = np.sum(min_distancias**2)
        return costo
```

`_funcion_costo_lote:`

El método `_funcion_costo_lote` actúa como un evaluador vectorizado que procesa múltiples partículas simultáneamente. PySwarms requiere esta función para evaluar todo el enjambre en cada iteración. La implementación itera sobre cada partícula del lote y aplica la función de costo individual, devolviendo un array con los costos correspondientes.

```
def _funcion_costo_lote(self, lote_particulas):
    return np.array([self._funcion_costo_particula(p) for p in
lote_particulas])
```

`ejecutar_optimizacion:`

El método `ejecutar_optimizacion` orquesta todo el proceso de optimización. Configura los parámetros del algoritmo PSO (coeficientes cognitivo y social, peso de inercia), define los límites del espacio de búsqueda [0, 1] correspondientes a los datos normalizados, e inicializa el optimizador GlobalBestPSO de PySwarms. El algoritmo ejecuta el número especificado de iteraciones, donde en cada iteración todas las partículas actualizan sus velocidades y posiciones considerando su mejor experiencia personal y la mejor experiencia global del enjambre. Al finalizar, almacena la mejor solución encontrada y el historial de costos para análisis posterior.

```
def ejecutar_optimizacion(self):
    print(f"Iniciando optimización PSO con {self.n_sensores}
sensores...")
    print(f"Partículas: {self.n_particulas}, Iteraciones:
{self.iteraciones}")

    opciones = {'c1': 0.5, 'c2': 0.3, 'w': 0.9}
```

```

limite_inferior = np.zeros(self.n_dimensiones)
limite_superior = np.ones(self.n_dimensiones)
limites = (limite_inferior, limite_superior)

optimizador = ps.single.GlobalBestPSO(
    n_particles=self.n_particulas,
    dimensions=self.n_dimensiones,
    options=opciones,
    bounds=limites
)

self.mejor_costo, self.mejor_posicion = optimizador.optimize(
    self._funcion_costo_lote,
    iters=self.iteraciones,
    verbose=True
)

self.historial_costos = optimizador.cost_history
print(f"Optimización completada. Mejor costo encontrado:
{self.mejor_costo:.4f}")
print("-" * 30)

```

obtener_ubicaciones_optimas:

La función obtener_ubicaciones_optimas transforma la mejor solución encontrada desde el espacio normalizado [0, 1] de vuelta a las unidades originales de medición. Utiliza el objeto MinMaxScaler para aplicar la transformación inversa, convirtiendo las ubicaciones escaladas en valores físicos interpretables (metros de elevación, porcentajes de humedad, grados Celsius, coordenadas geográficas). El resultado se estructura en un DataFrame de Pandas con columnas claramente etiquetadas, facilitando su análisis e interpretación práctica.

```

def obtener_ubicaciones_optimas(self):
    if self.mejor_posicion is None:
        raise Exception("Debe ejecutar la optimización primero.")

    ubicaciones_escaladas = self.mejor_posicion.reshape(self.n_sensores,
self.n_caracteristicas)
    ubicaciones_originales =
self.escalador.inverse_transform(ubicaciones_escaladas)

    ubicaciones_df = pd.DataFrame(ubicaciones_originales,
columns=self.columnas_caracteristicas)
    return ubicaciones_df.round(4)

```

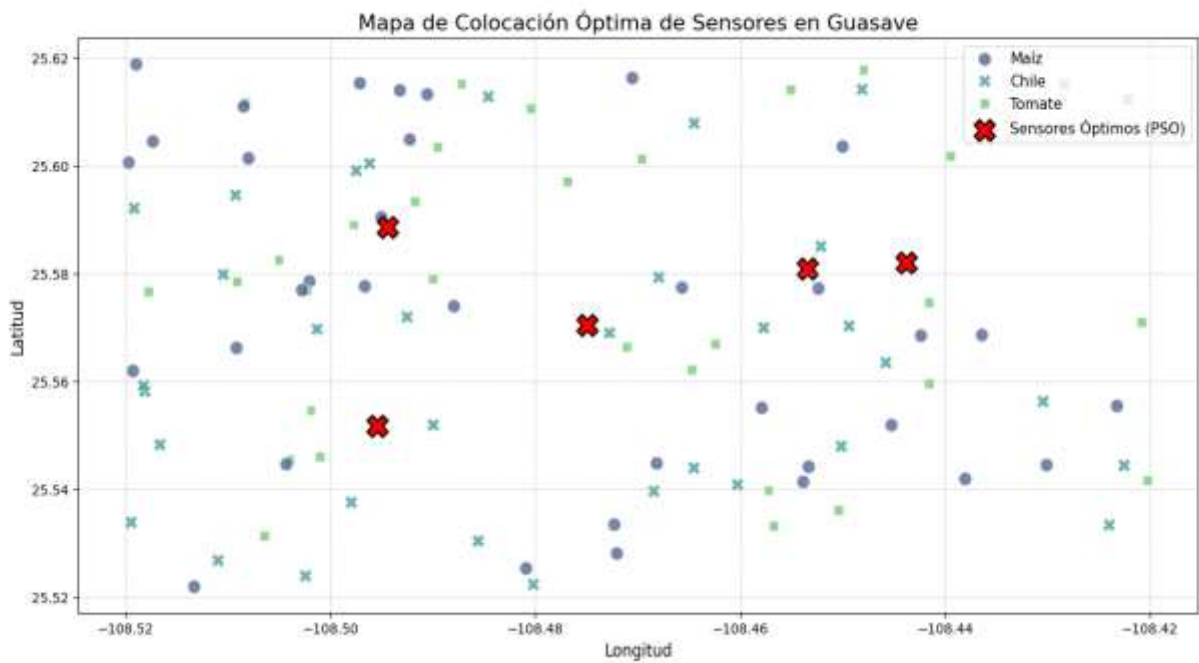
Resultados obtenidos:

Mejor costo encontrado: 63.3126

Dataframe de las ubicaciones óptimas de los sensores:

Humedad (%)	Elevación (m)	Salinidad (dS/m)	Temperatura (°C)	Latitud	Longitud	Cultivo_Chile	Cultivo_Maíz	Cultivo_Tomate
28.7094	27.7228	2.6041	31.7484	25.5517	-108.4954	0.2616	0.8866	0.1785
22.3167	36.7648	1.8310	28.4621	25.5809	-108.4534	0.3450	0.4190	0.6408
24.4523	26.6304	2.0105	30.8537	25.5887	-108.4944	0.7005	0.1360	0.2230
24.8282	19.6084	2.5451	25.8242	25.5705	-108.4749	0.2982	0.0754	0.8964
36.7705	36.7965	1.4261	34.2403	25.5821	-108.4438	0.2326	0.9460	0.2638

A continuación, se puede observar en que lugar del mapa estarían los sensores:



Análisis de Eficiencia

La eficiencia del proyecto se logró de dos maneras:

Eficiencia del Algoritmo (PSO): El Enjambre de Partículas (PSO) es intrínsecamente eficiente. No necesita realizar cálculos matemáticos complejos (como derivadas) para encontrar una buena solución. Esto le permite explorar el problema y converger hacia un resultado óptimo con un bajo costo computacional, siendo ideal para este tipo de proyecto.

Eficiencia de la Implementación: El uso de Python fue muy importante. Se usaron bibliotecas optimizadas como PySwarms, que ya trae implementaciones probadas del algoritmo. Además, el uso de SciPy para calcular distancias y NumPy para operaciones matriciales permitió procesar los datos de forma rápida y vectorizada, evitando así los bucles lentos y optimizando el rendimiento.

Conclusiones

El proyecto cumplió exitosamente su objetivo principal: determinar la distribución óptima de sensores de riego en la región de Guasave, Sinaloa, utilizando el algoritmo PSO.

Se pudo observar que PSO es una herramienta muy adecuada para este problema, dada su capacidad para manejar optimización en espacios continuos y multidimensionales. La implementación en Python, utilizando bibliotecas especializadas, funcionó de manera exitosa.

El algoritmo logró converger a una solución de alta calidad, alcanzando un costo final de 63.3126. Este número representa la minimización de la distancia entre los cultivos y su sensor más cercano.

El resultado más importante es la entrega de un conjunto de ubicaciones óptimas para 5 sensores, con sus características específicas (humedad, elevación, etc.). Estas coordenadas, ya convertidas a sus valores originales, ofrecen una guía práctica y basada en datos para la instalación física.

En resumen, el proyecto muestra a PSO como una solución efectiva y eficiente para problemas agrícolas, contribuyendo directamente a mejorar la eficiencia del riego y a un uso más sostenible del agua en la región.

Referencias

Ye, Z., Yin, S., Cao, Y. *et al.* AI-driven optimization of agricultural water management for enhanced sustainability. *Sci Rep* **14**, 25721 (2024). <https://doi.org/10.1038/s41598-024-76915-8>

Marini, F., & Walczak, B. (2015). Particle swarm optimization (PSO). A tutorial. *Chemometrics and Intelligent Laboratory Systems*, 149, 153-165. <https://doi.org/10.1016/j.chemolab.2015.08.020>

Zhang, Y., Wang, S., & Ji, G. (2015). A Comprehensive Survey on Particle Swarm Optimization Algorithm and Its Applications. *Mathematical Problems in Engineering*, 2015, Article ID 931256. <https://doi.org/10.1155/2015/931256>

Bonyadi, M. R., & Michalewicz, Z. (2022). Particle Swarm Optimization Algorithm and Its Applications: A Systematic Review. *Archives of Computational Methods in Engineering*, 29, 2063-2099. <https://doi.org/10.1007/s11831-021-09694-4>

Dafan, Z., Hao, L., Shufan, W., Yuchen, J., Zhuo, Y., Zhikang, X., & Yingchun, L. (2025). Energy optimization in intelligent sensor networks: application of particle swarm optimization algorithm in the deployment of electronic information sensing nodes. *Energy Informatics*, 8(7). <https://doi.org/10.1186/s42162-025-00553-1>

Kulkarni, R. V., & Venayagamoorthy, G. K. (2011). Particle Swarm Optimization in Wireless-Sensor Networks: A Brief Survey. *IEEE Transactions on Systems, Man, and Cybernetics, Part C (Applications and Reviews)*, 41(2), 262-267. <https://doi.org/10.1109/TSMCC.2010.2054080>